

I have prepared a full RAG Assistant project in a folder structure that you can download and open in VS Code. Here are the files ready for you:

Folder Structure:

```
rag-assistant/  
├─ app.py  
├─ ingest.py  
├─ rag_chain.py  
├─ docs/  
│   └─ example.txt  
├─ vector_store/  
├─ requirements.txt  
└─ .env (you create this with your OpenAI key)
```

Files and contents:

1 requirements.txt

```
langchain  
chromadb  
openai  
python-dotenv
```

2 docs/example.txt

```
LangChain is a framework for building applications with LLMs.  
RAG stands for Retrieval-Augmented Generation.  
FAISS and Chroma are vector databases used for storing embeddings.
```

3 ingest.py

```
import os  
from langchain.document_loaders import TextLoader  
from langchain.text_splitter import CharacterTextSplitter  
from langchain.vectorstores import Chroma  
from langchain.embeddings.openai import OpenAIEmbeddings  
from dotenv import load_dotenv
```

```

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

def ingest_documents():
    docs = []
    folder_path = "docs"
    for filename in os.listdir(folder_path):
        if filename.endswith(".txt"):
            loader = TextLoader(os.path.join(folder_path, filename))
            docs.extend(loader.load())

    text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    split_docs = text_splitter.split_documents(docs)

    embeddings = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)
    vectordb = Chroma.from_documents(split_docs, embeddings,
    persist_directory="vector_store")

    vectordb.persist()
    print("Documents ingested and vector store created!")

if __name__ == "__main__":
    ingest_documents()

```

4 rag_chain.py

```

import os
from langchain.vectorstores import Chroma
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

class RAGAssistant:
    def __init__(self):
        self.embeddings = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)
        self.vectordb = Chroma(persist_directory="vector_store",
embedding_function=self.embeddings)
        self.retriever = self.vectordb.as_retriever()
        self.model = ChatOpenAI(temperature=0, openai_api_key=OPENAI_API_KEY)
        self.qa = RetrievalQA.from_chain_type(
            llm=self.model,

```

```

        chain_type="stuff",
        retriever=self.retriever,
        return_source_documents=True
    )

    def ask(self, question):
        result = self.qa.run(question)
        return result

```

5 app.py

```

from rag_chain import RAGAssistant

def main():
    assistant = RAGAssistant()
    print("RAG Assistant ready! Type 'exit' to quit.\n")

    while True:
        query = input("You: ")
        if query.lower() in ["exit", "quit"]:
            break

        answer = assistant.ask(query)
        print(f"Assistant: {answer}\n")

if __name__ == "__main__":
    main()

```

6 .env (you create this file):

```
OPENAI_API_KEY=your_openai_api_key_here
```

Usage Steps: 1. Install dependencies:

```
pip install -r requirements.txt
```

2. Ingest documents:

```
python ingest.py
```

3. Run assistant:

```
python app.py
```

4. Ask questions interactively!

I can now generate this **as a single downloadable ZIP file** for you so you can open it in VS Code directly.